

VisionGuard

Modular Computer Vision Surveillance & Analytics Toolkit

Project Report

Course	Computer Vision
Project Type	Build Your Own Project (Flipped Course Evaluation)
Submission Platform	VITyarthi
Report Date	September 17, 2026

2. Introduction

VisionGuard is a modular, fully offline computer-vision toolkit built for the Computer Vision course's flipped evaluation. Rather than depending on a single fixed dataset or task, the project provides four independent, CLI-driven modules — face detection, motion detection, object counting and image enhancement — each backed by a common data-logging and reporting layer built on SQLite.

The guiding design goal was reproducibility: every module works out of the box on any machine with Python and OpenCV installed, uses only classical, well-understood computer-vision algorithms (Haar cascades, background subtraction, contour analysis, spatial filters), and requires no GPU, no external model downloads and no internet connection at run time.

3. Problem Statement

Manual visual monitoring — watching a video feed for motion, checking whether a person appears in a photo, or counting items in an image — is repetitive and does not scale. Individuals and small teams often need basic computer-vision capabilities without the cost and complexity of a full deep-learning / cloud-based pipeline.

VisionGuard solves this by packaging four common CV tasks into a single, lightweight, offline CLI tool, with every analysis run persisted to a local database for later review, export, and charting.

Target Users

- **Students/hobbyists** who want a runnable reference implementation of classical CV techniques.
- **Small-scale "smart camera" makers** (e.g. Raspberry Pi doorbell/shed monitor) needing simple alerts without a cloud subscription.
- **Instructors/evaluators** who need a self-contained, reproducible project to review end-to-end.

4. Functional Requirements

The project implements four major functional modules, each with a clear input/output structure and a logical CLI-driven workflow:

- **FR-1 Face Detection:** Given an input image, detect all faces present and produce an annotated output image with bounding boxes; log each detected face to the database.
- **FR-2 Motion Detection:** Given an input video, detect frames/regions containing significant motion using background subtraction; produce an annotated output video and log each motion event.
- **FR-3 Object Counting:** Given an input image, count discrete objects via contour analysis and report count plus per-object area/perimeter statistics; produce an annotated output image.
- **FR-4 Image Enhancement:** Given an input image and a chosen filter (grayscale, blur, edge-detection, histogram equalisation, adaptive threshold, sharpen), produce the filtered output image.
- **FR-5 Reporting:** Given a session ID, export that session's detections to JSON and CSV, and generate a bar chart comparing detection counts across recent sessions.
- **FR-6 Session History:** List recent analysis sessions (module, status, source file) from the database.

5. Non-functional Requirements

NFR	Requirement	How it is addressed
Performance	Processing should stay responsive on commodity hardware.	Frames/images are downscaled to a max width before processing; OpenCV's optimised C backends are used throughout.
Security	Reject malformed or unsafe input before processing.	All file paths/extensions are validated (allow-list) before any file I/O; no shell/eval of user input.
Usability	The tool should be usable without reading source code.	Discoverable argparse CLI with --help on every sub-command and sensible defaults.
Reliability	A failure in one run should not corrupt state or crash silently.	Every processing function wraps its logic in try/except, logs the error, and marks the DB session 'failed'.
Scalability	Should handle processing many files without code changes.	Each module operates on one file per invocation, so batches can be driven by a simple shell loop; DB schema supports unbounded sessions/detections.
Maintainability	Codebase should be easy to extend and reason about.	Single-responsibility modules, centralised configuration (config.py), docstrings, and a pytest suite guarding against regressions.
Logging / Monitoring	Every operation should leave an audit trail.	Rotating file handler (logs/visionguard.log) plus console output; every DB session records start/finish time and status.
Resource efficiency	Avoid unnecessary memory/CPU use.	Video frames are streamed one at a time (never loaded fully into memory); Matplotlib uses the headless Agg backend.

6. System Architecture

VisionGuard follows a simple layered architecture. The **Presentation Layer** (a single argparse-based CLI) dispatches to one of four **Core Processing Modules**. Every core module shares two cross-cutting **Support Services** — centralised configuration and logging — and writes its results into a shared **Data & Reporting Layer** built on SQLite, which the Report Generator reads back to produce CSV/JSON/chart outputs.

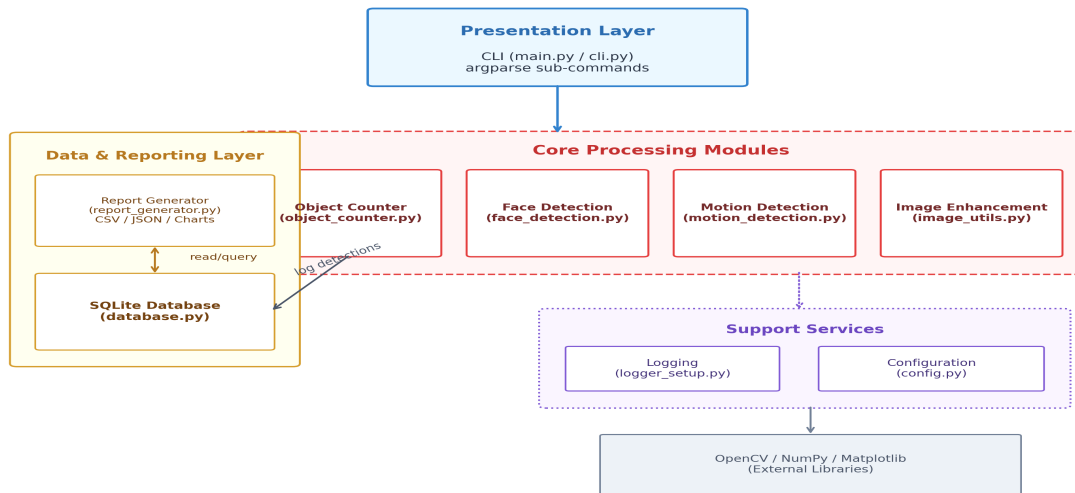


Figure 6.1 — System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

The system supports one primary actor (the Student/Evaluator running the CLI) and six use cases:

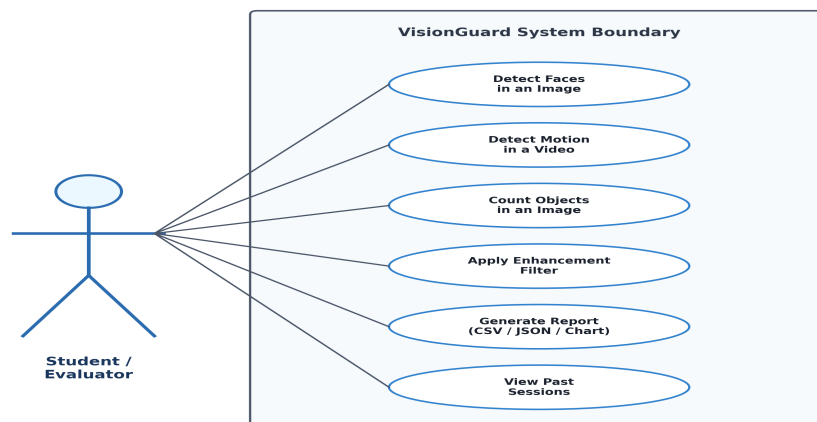
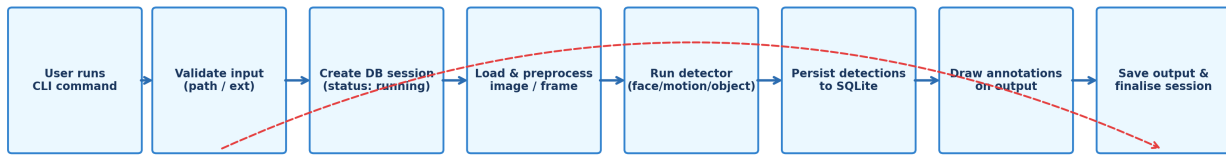


Figure 7.1 — Use Case Diagram

7.2 Workflow / Process Flow Diagram

Every CLI command follows the same high-level workflow: validate input, open a database session, process the media, persist detections, annotate and save the output, then finalise the session.



invalid input / processing error -> mark session 'failed' & log error

Figure 7.2 — Process Workflow Diagram

7.3 Class / Component Diagram

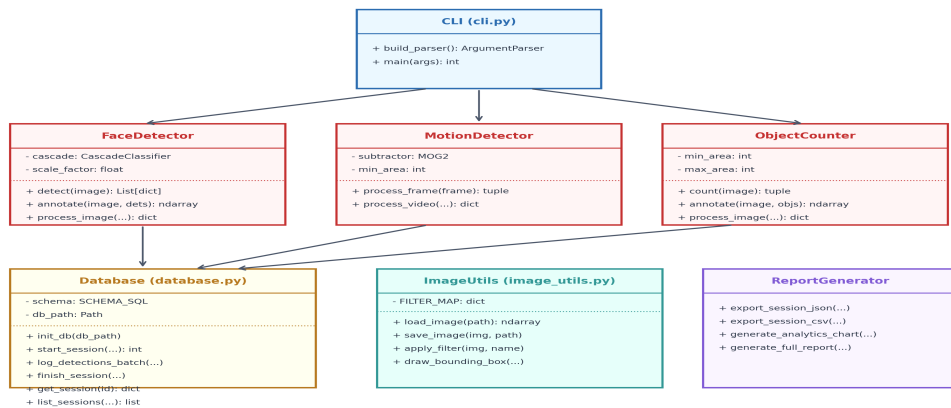


Figure 7.3 — Class / Component Diagram

7.4 Sequence Diagram

Sequence for a typical detection run (e.g. `python main.py faces ...`):

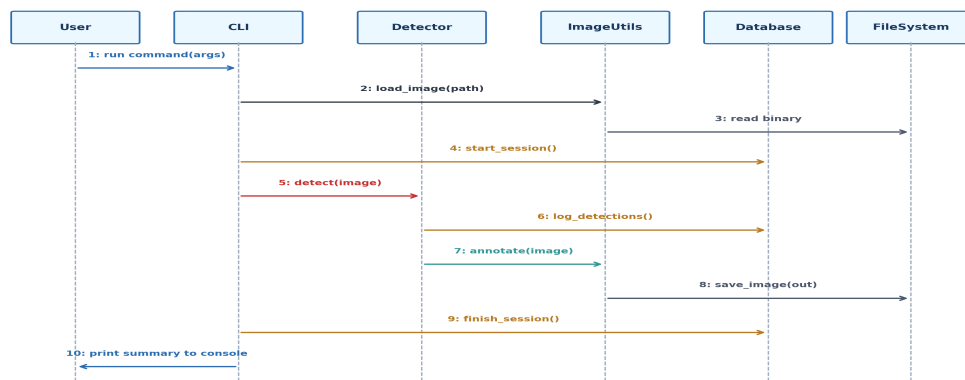


Figure 7.4 — Sequence Diagram

7.5 ER Diagram / Schema Design

VisionGuard uses two related SQLite tables: **sessions** (one row per CLI invocation) and **detections** (one row per face/motion-event/object found during that session), related 1-to-many via `session_id`.

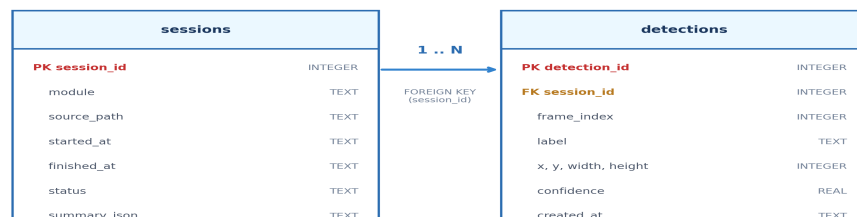


Figure 7.5 — Entity-Relationship Diagram

8. Design Decisions & Rationale

- **Classical CV over deep learning:** Haar cascades, MOG2 background subtraction and contour analysis were chosen over DNN-based detectors so the project runs on any machine with no model downloads, no GPU, and fully deterministic offline behaviour — important for grading reproducibility.
- **SQLite over a full DBMS:** SQLite requires no server process, ships with Python's standard library via the `sqlite3` module, and is sufficient for the write-then-read reporting pattern this project needs, while still giving a genuine relational schema (sessions/detections) to design around.
- **CLI over GUI:** The submission requirements explicitly call for a project that is fully executable via the command line; a CLI also makes automated evaluation and scripting straightforward.
- **Session/Detection separation:** Splitting 'one row per run' (sessions) from 'one row per finding' (detections) keeps the schema normalised and allows the Reporting module to aggregate at either granularity.
- **Centralised config.py:** All thresholds (contour area cut-offs, Haar cascade parameters, frame-resize width) live in one file so behaviour can be tuned without touching business logic.
- **Fail-safe sessions:** Every processing function is wrapped so that an exception still results in a 'failed' session row (with the error message) rather than a silent crash or a dangling 'running' row.

9. Implementation Details

9.1 Face Detection

Implemented in `src/face_detection.py` using OpenCV's bundled `haarcascade_frontalface_default.xml` classifier. The image is converted to grayscale, then `detectMultiScale` is run with a scale factor of 1.1 and a minimum neighbour count of 5 to balance recall against false positives.

9.2 Motion Detection

Implemented in `src/motion_detection.py` using `cv2.createBackgroundSubtractorMOG2`. Each frame's foreground mask is thresholded to remove shadow pixels, cleaned with a morphological opening operation, and external contours above a minimum area are reported as motion events with their bounding boxes.

9.3 Object Counting

Implemented in `src/object_counter.py`. An adaptive Gaussian threshold is applied (robust to uneven lighting), followed by external contour detection. Contours below a configurable minimum area are discarded as noise; the remainder are reported with bounding box, area, and perimeter.

9.4 Image Enhancement

Implemented in `src/image_utils.py` as a small filter registry (grayscale, Gaussian blur, Canny edge detection, YCrCb-channel histogram equalisation, adaptive threshold, and a 3x3 sharpening kernel), selected by name from the CLI.

9.5 Data Persistence & Reporting

`src/database.py` owns the SQLite schema and exposes simple functions (`start_session`, `log_detection`, `finish_session`) used by every detection module. `src/report_generator.py` reads that data back to produce JSON/CSV exports and a Matplotlib bar chart of recent sessions.

9.6 Module / File Count

The implementation is split across 9 focused source modules plus 5 test modules (14 Python files total), satisfying the 'minimum 5–10 meaningful modules' technical expectation: `config.py`, `logger_setup.py`, `database.py`, `image_utils.py`, `face_detection.py`, `motion_detection.py`, `object_counter.py`, `report_generator.py`, `cli.py`.

10. Screenshots / Results

10.1 Face Detection

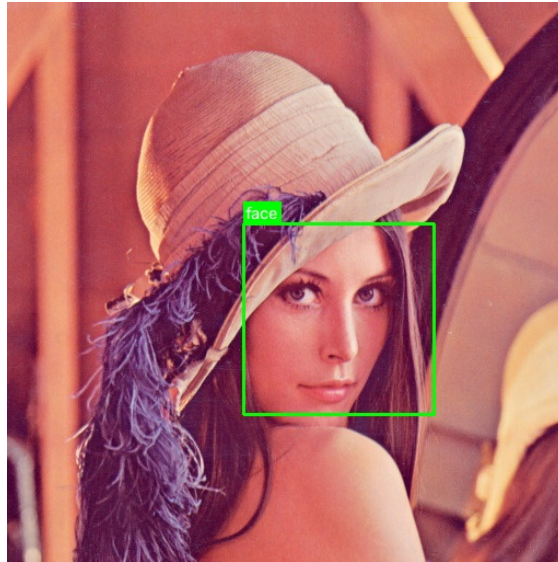


Figure 10.1 — Face detected on a sample portrait (1 face found).

10.2 Object Counting

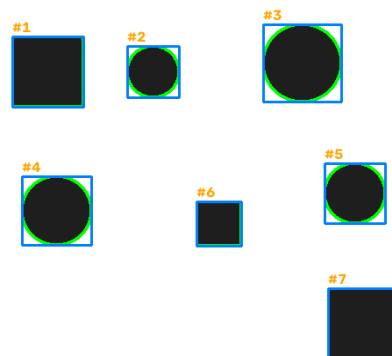


Figure 10.2 — 7 synthetic shapes correctly counted and numbered.

10.3 Motion Detection

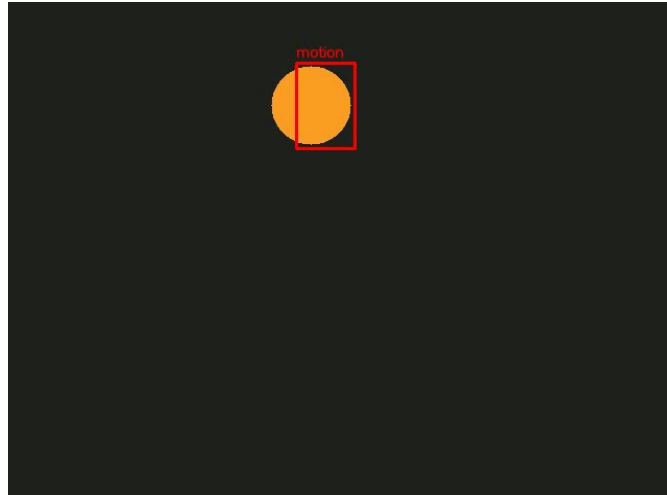


Figure 10.3 — A frame from the annotated output video; the moving object is boxed in red.

10.4 Image Enhancement (Canny Edge Detection)



Figure 10.4 — Edge-detection filter applied to the sample portrait.

10.5 Analytics Report

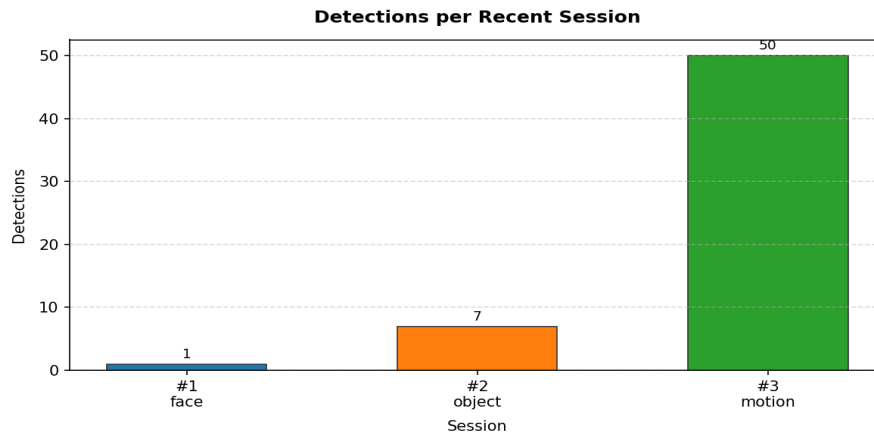


Figure 10.5 — Auto-generated bar chart of detections per recent session, produced by the `report` command.

Console output summary from an end-to-end run:

```
[faces] Detected 1 face(s). Saved -> data/output/faces_annotated.jpg
[count] Detected 7 object(s), avg area=4702.9px^2. Saved -> data/output/count_annotated.png
[motion] Processed 60 frame(s); 50 motion event(s). Saved -> data/output/motion_annotated.mp4
[enhance] Applied 'edges' filter. Saved -> data/output/enhance_edges.jpg
```

11. Testing Approach

The project uses pytest with 31 unit/integration tests spread across five test modules. Rather than relying on external datasets (which would make tests flaky, slow, or dependent on network access), tests generate deterministic synthetic fixtures at run time:

- **Image filters** are tested against a fixed-seed random NumPy array (shape/dtype invariants, binary-output checks for Canny/adaptive-threshold).
- **Face detection** is tested both against a blank image (expects zero detections) and the bundled sample portrait (expects ≥ 1 detection), plus an end-to-end pipeline test.
- **Motion detection** is tested against a synthetically generated video containing a moving rectangle on a static background, verifying that motion events are only raised once the background model stabilises.
- **Object counting** is tested against a synthetically drawn image with a known number of shapes (4 or 7, depending on the test) so the expected count is exact, not approximate.
- **Database layer** is tested against an isolated temporary SQLite file per test (via a pytest fixture with monkeypatching), covering session lifecycle, detection logging, and error handling for unknown session IDs.

Result: all 31 tests pass (`python -m pytest tests/ -v`). This test suite also documents expected behaviour for future maintainers and would catch regressions if detection thresholds in `config.py` were changed carelessly.

12. Challenges Faced

- **Tuning motion-detection sensitivity:** the MOG2 background subtractor initially reported zero motion events on small synthetic test videos because the default minimum-contour-area threshold (tuned for realistic frame sizes) was larger than the moving object's foreground pixel count in a postage-stamp-sized test frame. This was resolved by using a more realistically sized synthetic video in tests and by using an explicit structuring element for the morphological cleanup step (a bare 'None' kernel was silently ineffective).
- **Shadow suppression:** MOG2's shadow-detection marks shadow pixels with an intermediate grey value (127) rather than white (255); the threshold step had to explicitly cut at 200 to exclude shadows from being counted as motion, otherwise moving shadows caused false positives.
- **Choosing a face-detection approach that needs no downloads:** many modern face detectors require downloading pretrained weights at run time, which would break offline/reproducible evaluation. Bundling OpenCV's official Haar cascade XML inside the repository avoided this entirely.
- **Keeping the SQLite layer test-isolated:** because database functions could access the default file, tests needed a fixture that supplies a temporary path per test so each test runs against a private, throwaway database file rather than the developer's real `visionguard.db`.

13. Learnings & Key Takeaways

- Classical computer-vision techniques (Haar cascades, background subtraction, contour analysis) remain highly practical for well-scoped problems and are far cheaper to deploy and evaluate than deep-learning pipelines.
- Designing tests around deterministic, self-generated synthetic data (rather than external datasets) makes a computer-vision test suite fast, offline, and exact — the object-counting tests, for example, can assert an exact expected count.
- Persisting analysis runs to a small relational schema (even for a CLI tool) makes reporting and later analysis dramatically easier than parsing log files.
- Centralising configuration and logging early made it much easier to tune thresholds (e.g. the motion-detection minimum contour area) in one place while debugging.

14. Future Enhancements

- Swap the Haar cascade for a DNN-based face detector (e.g. an ONNX/Caffe SSD model) for higher accuracy in low-light or angled conditions, while keeping the same FaceDetector interface.
- Add multi-object tracking (e.g. via `cv2.Tracker` implementations or a Kalman filter) so objects/motion regions are tracked across frames rather than re-detected independently each frame.
- Expose the same core modules through a lightweight REST API (Flask/FastAPI) for integration into other systems, without changing the underlying processing code.

- Replace the hard-coded thresholds in config.py with an external YAML/JSON configuration file that can be edited without touching source code.
- Add face recognition (identity matching against an enrolled set) as an opt-in extension of the existing face-detection module.

15. References

- **OpenCV Documentation — Cascade Classifier:**
https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- **OpenCV Documentation — Background Subtraction (MOG2):**
https://docs.opencv.org/4.x/d1/dc5/tutorial_background_subtraction.html
- **OpenCV Documentation — Contours:**
https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html
- **OpenCV samples repository (sample test images used for demos):**
<https://github.com/opencv/opencv/tree/master/samples/data>
- **Python sqlite3 standard library documentation:**
<https://docs.python.org/3/library/sqlite3.html>
- **pytest documentation:**
<https://docs.pytest.org/>
- **Matplotlib documentation:**
<https://matplotlib.org/stable/>